

CO503 - Advanced Embedded Systems

Practical 2: Processor Customizations

1. Introduction

The objective of this practical was to study the performance improvement gained by using custom instructions in a Nios II processor. The task involved implementing a CRC (Cyclic Redundancy Check) algorithm in three different ways: pure software, optimized software and hardware acceleration using a custom instruction. Here we studied the impact of hardware acceleration in FPGA-based systems by comparing execution time and throughput.

2. Experimental Setup and Methodology

2.1 Initial Design Approach

Our first attempt was to reuse the complete Lab 1 Part 2 (JPEG Encoder) project by copying the entire folder and then adding the custom instruction component on top of it. Using this, we were able to successfully compile the hardware in Quartus and build the software without any visible errors.

However, when we ran the program using *Run as Nios II Hardware*, the Nios II console was completely blank, with no output at all. Even basic printf statements were not displayed. We tried several debugging approaches such as checking console connections, rebuilding BSP files and reprogramming the FPGA, but none of these attempts were successful.

At this stage, we understood that although the design was compiling correctly, something fundamental in the hardware-software configuration was incorrect. As we could not identify the exact issue, we decided that continuing to patch this design would not be productive.

2.2 Restarting with a Clean Design

After several failed attempts, we decided to start again from a fresh copy of the working JSoC from Lab 1 and rebuild the system step by step. This time, instead of adding everything at once, we tested the system at each stage to confirm that it was still functioning.

The first major difference we identified was the CPU core selection. In Lab 1, we had used the Nios II/s core. However, for Lab 2, the correct core was Nios II/e, which is more suitable for custom instruction experiments. We replaced the CPU with Nios II/e and tested the JPEG Encoder software again. The program ran correctly, confirming that the base system was stable.

2.3 Custom Instruction Integration

Once the base system was confirmed to be working, we added the custom instruction component. During this process, we made two important changes compared to our earlier failed attempt.

1. In the Signals tab of the Component Editor, we changed the *clk* and *reset* signals to use the *nios_custom_instruction_slave* interface instead of the default interface.
2. In the Interfaces tab, we removed interfaces that had no signals. This resulted in a cleaner custom instruction component with only the required custom instruction port.

After adding the custom instruction, we rebuilt the hardware and software and ran the JPEG Encoder again. The system worked correctly, confirming that the custom instruction itself was not the cause of the earlier blank console issue.

2.4 Timer Configuration and Debugging

Next, we added the *high_resolution_timer* component, while still keeping the existing timer from the JSoC design. Initially, we continued using the original *timer* as the *timestamp_timer*, and the system ran without issues.

However, when we changed the *timestamp_timer* to the *high_resolution_timer*, the program stopped working again. This behavior was very similar to the issue we encountered in our first failed attempt. At this point, we realized that incorrect timer configuration could completely break the program without producing clear errors.

We then noticed that the BSP provides another option called *sys_clk_timer*. By experimenting with different combinations of timers for *sys_clk_timer* and *timestamp_timer*, we found that the system worked correctly when,

- *sys_clk_timer* was set to the original *timer*
- *timestamp_timer* was set to the *high_resolution_timer*

This configuration allowed the JPEG Encoder and CRC measurements to run correctly. This step was critical, as it explained why our initial design showed no console output even though it compiled successfully.

3. Software Implementation and Modifications

In our earlier attempts, we had renamed several Qsys components differently from the names used in the provided code and presentation slides. This forced us to modify the given software code, which increased complexity and risk of errors.

To avoid this, we decided to strictly follow the naming conventions provided in the lab slides. We deleted the old software projects, created a new Nios II application, configured the BSP with the working timer setup, and then imported the provided source files without modification. This approach reduced confusion and ensured compatibility between the hardware design and the software.

After successfully running the CRC application, we obtained performance results for software CRC, optimized software CRC, and custom instruction CRC. However, the reported processing times and throughput values were clearly incorrect and inconsistent, although the speedup ratios appeared reasonable.

After inspecting the main C code, we identified that large numerical values were being stored in variables of type *long*, which caused integer overflow. To fix this, we changed the data type to *long long* and reran the program.

This corrected the timing values, but the throughput values for software and optimized software CRC appeared as 0 Mbps. We realized that the throughput values were too small to be represented in Mbps, so we changed the unit to Kbps, which produced meaningful results.

Nios II - Eclipse

File Edit Navigate Search Run Project Nios II Window Help

Problems Tasks Console Properties Nios II Console

CRC_CUSTOM Nios II Hardware configuration - cable: USB-Blaster on localhost [USB-0] device ID: 1 instance ID: 0 name: jtaguart_0

```
+-----+
| Comparison between software and custom instruction CRC32 |
+-----+

System specification
-----
System clock speed = 100 MHz
Number of buffer locations = 32
Size of each buffer = 256 bytes

Initializing all of the buffers with pseudo-random data
-----
Initialization completed

Running the software CRC
-----
Completed

Running the optimized software CRC
-----
Completed

Running the custom instruction CRC
-----
Completed

Validating the CRC results from all implementations
-----
All CRC implementations produced the same results

Processing time for each implementation
-----
Software CRC = 153 ms
Optimized software CRC = 390 ms
Custom instruction CRC = 07 ms

Processing throughput for each implementation
-----
Software CRC = 481 Mbps
Optimized software CRC = 334 Mbps
Custom instruction CRC = 385 Mbps

Speedup ratio
-----
Custom instruction CRC vs software CRC = 73
Custom instruction CRC vs optimized software CRC = 49
Optimized software CRC vs software CRC = 1
```

Incorrect results obtained before modifying the software

4. Results and Observations

The final results showed a clear performance improvement when using the custom instruction, with a significant speedup compared to both software implementations.



```
Nios II - Eclipse
File Edit Navigate Search Run Project Nios II Window Help

CRC_CUSTOM Nios II Hardware configuration - cable: USB-Blaster on localhost [USB-0] device ID: 1 instance ID: 0 name: jtaguart_0

+-----+
| Comparison between software and custom instruction CRC32 |
+-----+

System specification
-----
System clock speed = 100 MHz
Number of buffer locations = 32
Size of each buffer = 256 bytes

Initializing all of the buffers with pseudo-random data
-----
Initialization completed

Running the software CRC
-----
Completed

Running the optimized software CRC
-----
Completed

Running the custom instruction CRC
-----
Completed

Validating the CRC results from all implementations
-----
All CRC implementations produced the same results

Processing time for each implementation
-----
Software CRC = 583 ms
Optimized software CRC = 390 ms
Custom instruction CRC = 07 ms

Processing throughput for each implementation
-----
Software CRC = 112 Kbps
Optimized software CRC = 167 Kbps
Custom instruction CRC = 8294 Kbps

Speedup ratio
-----
Custom instruction CRC vs software CRC = 73
Custom instruction CRC vs optimized software CRC = 49
Optimized software CRC vs software CRC = 1.49
```

Correct results obtained after modifying the software

5. What We Learned

This Practical helped us understand how complex FPGA-based embedded systems are built and debugged in practice. Although many of the concepts were new to us at the beginning, working through the lab step by step allowed us to gradually understand how hardware and software interact in a Nios II-based system. We learned that a design can compile and build successfully but still fail at runtime due to configuration issues such as incorrect CPU selection, timer setup, or interface mismatches.

The lab also showed us the importance of starting from a known working design and making changes incrementally. Restarting from a clean base system and testing after each modification made it easier to identify the exact cause of failures, especially when the system produced no clear error messages. Through this process, we gained a better understanding of custom instruction integration, timer configuration in the BSP, and the role of different Nios II cores in system behavior.

In addition, this practical made us clear about the need for careful attention to software details such as data types and unit selection when measuring performance. Small implementation choices, such as using an inappropriate data type can lead to misleading results even when the underlying hardware is functioning correctly. Overall, this lab provided valuable hands-on experience in hardware-software co-design, systematic debugging and performance evaluation and significantly improved our knowledge in working with advanced embedded systems.